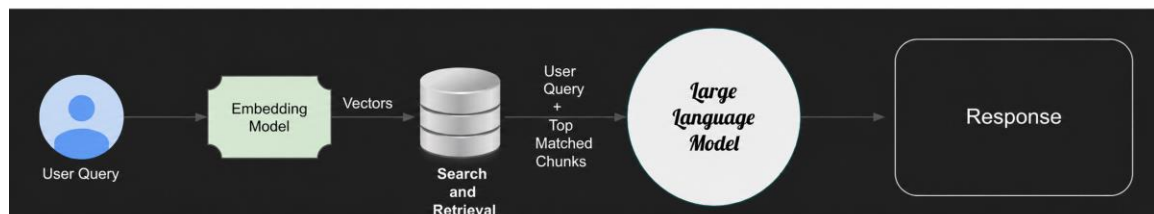
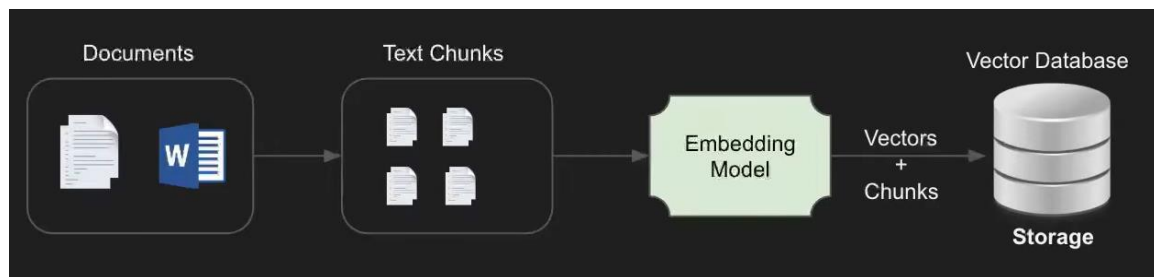


RAG-Based Symposium Chatbot using n8n

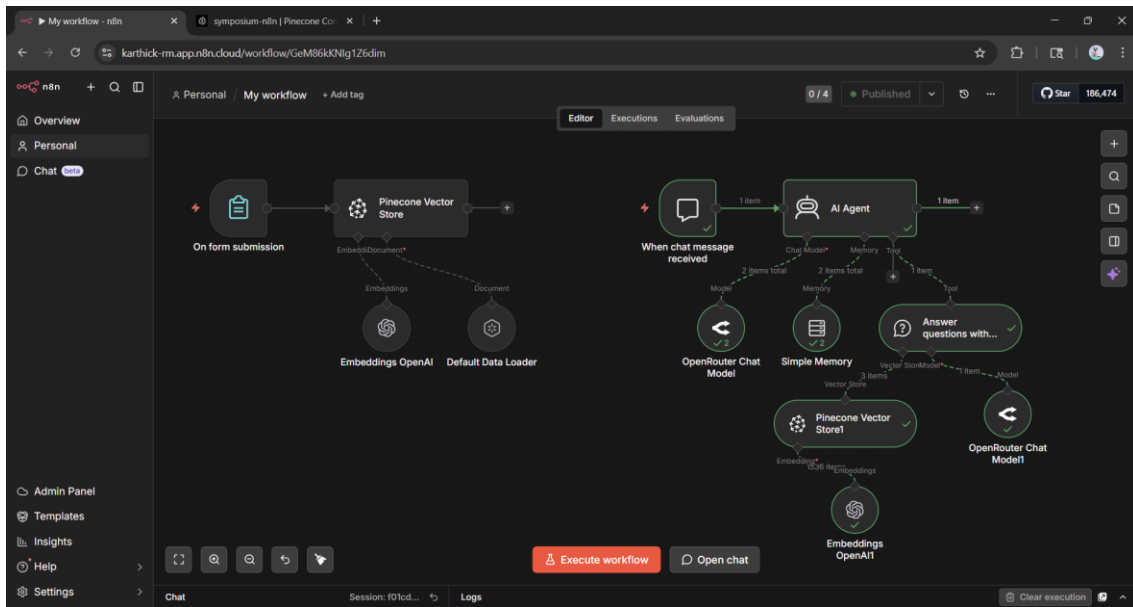
1. Introduction

This document provides a comprehensive explanation of a Retrieval-Augmented Generation (RAG) chatbot built using n8n. The system is designed to process a symposium-related PDF document and enable users to interact with them through a conversational interface. Unlike traditional chatbots that rely solely on pre-trained knowledge, this system grounds its responses in actual uploaded data, ensuring higher accuracy and relevance.

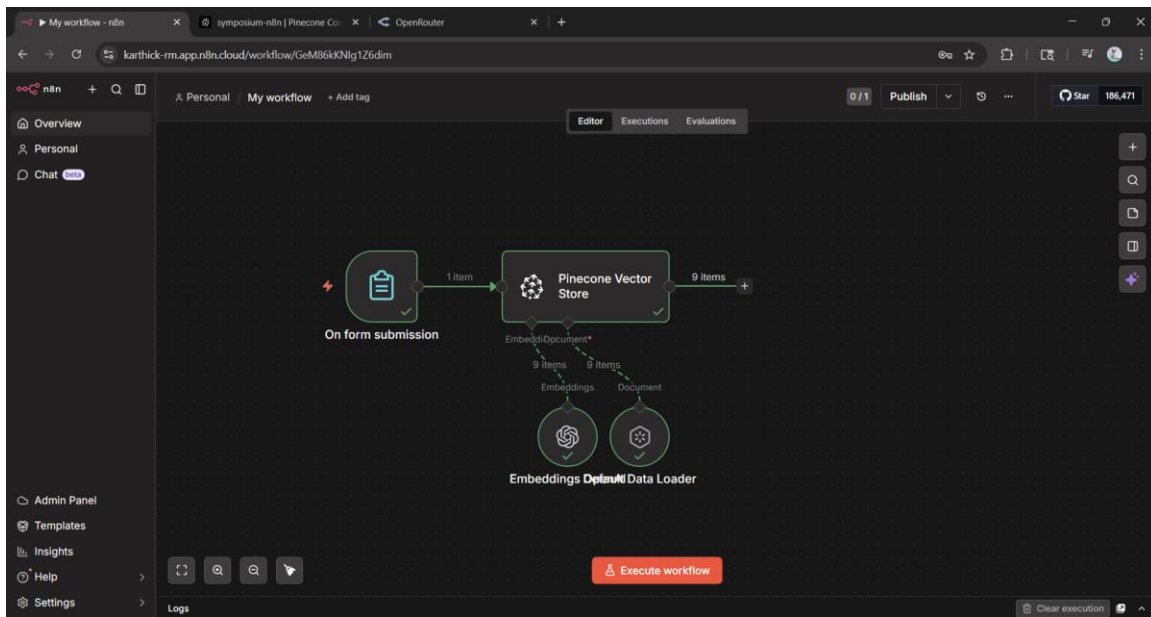


2. System Overview

The workflow is divided into two tightly coupled pipelines: the document ingestion pipeline and the query-response pipeline. The ingestion pipeline is responsible for accepting raw documents, extracting meaningful content, and transforming it into vector embeddings. The query-response pipeline processes user queries, retrieves relevant contextual data from the vector store, and generates precise answers using a language model.

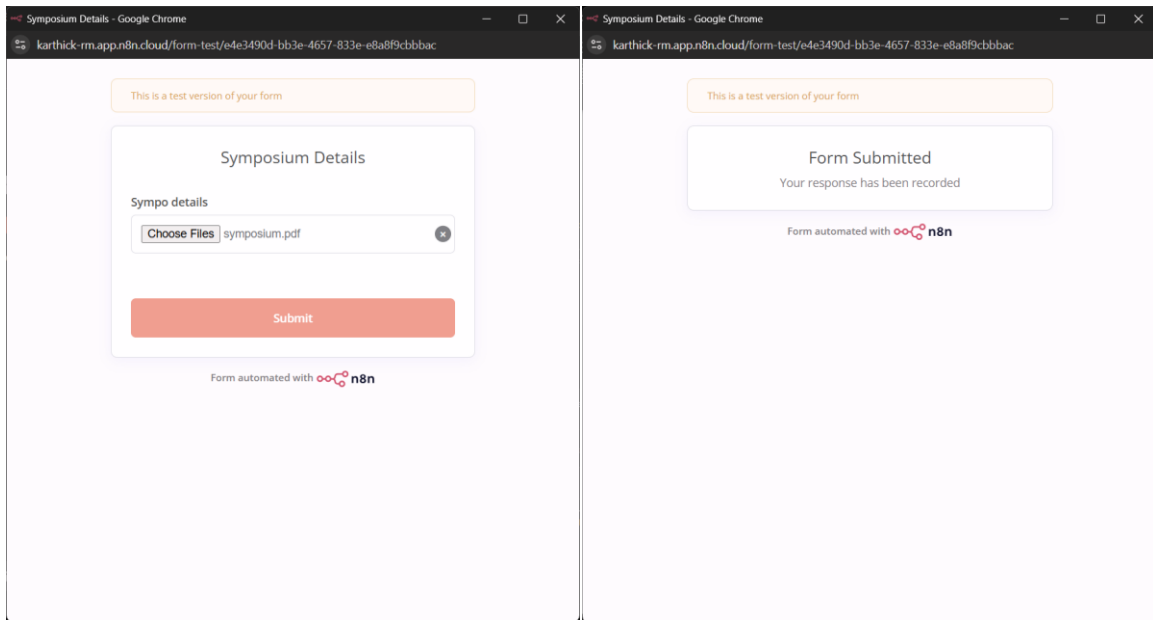


3. Document Ingestion Pipeline



3.1 Form Trigger Mechanism

The workflow begins with a form-based trigger that allows users to upload PDF documents containing symposium information. This component acts as the entry point for the system and ensures that incoming data is captured in a structured format suitable for further processing.



3.2 Document Parsing and Text Extraction

Once the files are received, they are passed to a document loader configured to handle PDF formats. The loader extracts textual content from the binary files and prepares it for downstream processing. At this stage, the content is divided into manageable chunks using a simple splitting strategy. This step is critical because smaller chunks improve retrieval accuracy during the querying phase.

3.3 Embedding Generation

Each text chunk is transformed into a numerical representation using *text-embedding-3-small* which is the embedding model used. These embeddings capture the semantic meaning of the text, enabling similarity-based search. The choice of a lightweight embedding model ensures faster processing while maintaining acceptable accuracy.

4.1 Chat Trigger and User Interaction

The second pipeline begins when a user sends a message through the chat interface. This trigger initializes the conversational flow and passes the query to the AI agent.

4.2 Role of the AI Agent

The AI agent acts as the central orchestrator of the system. It interprets user queries, decides when to invoke retrieval mechanisms, and composes the final response. It integrates multiple components including language models, memory, and retrieval tools.

Here, the *OpenRouter Chat model* is used that selects *google/gemini-2.0-flash-001* as LLM and *Simple Memory* as the memory model. For retrieval, the workflow uses a tool called “*Answer questions with a vector store*” that collects data from the vector store model Pinecone vector store that embeds data using *Embeddings OpenAI*. The workflow again uses the OpenRouter chat model(*google/gemini-2.0-flash-001*) for answering queries.

4.3 Conversational Memory

A short-term memory module is used to retain recent interactions. This allows the chatbot to maintain context across multiple turns, enabling more natural and coherent conversations.

4.4 Query Embedding and Similarity Search

When a query is received, it is converted into an embedding using the same model used during ingestion. This ensures consistency in vector representation. The system then performs a similarity search in the Pinecone database to identify the most relevant text chunks.

4.5 Context-Aware Response Generation

The retrieved chunks are passed to the language model along with the user query. The model uses this context to generate an informed and accurate response. This approach minimizes hallucination and ensures that answers are grounded in actual data.

5. End-to-End Data Flow

The system operates in two phases. In the first phase, documents are uploaded, processed into text, converted into embeddings, and stored. In the second phase, user queries are embedded, matched against stored vectors, and used to generate responses. This seamless integration of storage and retrieval enables the chatbot to function as an intelligent knowledge assistant.

6. Production link

<https://karthick-rm.app.n8n.cloud/webhook/301b4c20-0a78-419d-a3b9-8c402c395d61/chat>

7. Design Considerations

The system prioritizes efficiency and scalability. A lightweight embedding model is chosen to reduce computational cost, while limiting the number of retrieved results ensures faster response times. Short-term memory is used to balance contextual understanding with token efficiency.

8. Limitations

The performance of the system depends heavily on the quality of the input documents. Simple text splitting may not always capture semantic boundaries effectively. Additionally, the absence of advanced ranking mechanisms may occasionally result in suboptimal retrieval.

9. Future Enhancements

The system can be improved by incorporating advanced chunking strategies, reranking models, and metadata-based filtering. Expanding memory capabilities and integrating a user-friendly frontend interface would further enhance usability.

10. Conclusion

This RAG-based chatbot demonstrates how modern AI systems can combine retrieval mechanisms with language models to deliver accurate, context-aware responses. It serves as a scalable solution for managing and interacting with symposium-related knowledge.